

Building an Eisenhower Matrix App in C# on Linux

A step-by-step guide using Avalonia UI

Windows Forms only runs on Windows, so this guide uses **Avalonia UI**, a free, cross-platform GUI framework for C# that works well on Linux and is similar in spirit to Windows Forms and WPF.

The finished app has a text box for a task, two checkboxes (Important and Urgent), an Add button, and a 2×2 grid showing the four quadrants: **Do** (important + urgent), **Plan** (important + not urgent), **Delegate** (not important + urgent) and **Delete** (not important + not urgent). Double-clicking a task removes it.

Avalonia describes the window layout in a XAML file (**.axaml**) and keeps the logic in a C# file.

Step 1: Install the .NET SDK

Install .NET 8 or newer using your distro's package manager. On Ubuntu or Debian:

```
sudo apt install dotnet-sdk-8.0
```

On Fedora:

```
sudo dnf install dotnet-sdk-8.0
```

Package names can vary by distro and version. If yours doesn't have it, Microsoft's Linux install guide is at <https://learn.microsoft.com/dotnet/core/install/linux>. Check that it worked:

```
dotnet --version
```

For an editor, **VS Code** with the C# Dev Kit and Avalonia extensions works well, or **JetBrains Rider**, which is free for non-commercial use. A plain text editor and the terminal are also enough.

Step 2: Install the Avalonia templates and create the project

```
dotnet new install Avalonia.Templates
dotnet new avalonia.app -o EisenhowerMatrix
cd EisenhowerMatrix
```

This creates several files. You'll only edit two of them: **MainWindow.axaml** (the layout) and **MainWindow.axaml.cs** (the logic).

Step 3: Design the window in MainWindow.axaml

Replace the entire contents of **MainWindow.axaml** with this:

```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="EisenhowerMatrix.MainWindow"
        Title="Eisenhower Matrix"
        Width="900" Height="600"
        MinWidth="600" MinHeight="400">
```

```

        RequestedThemeVariant="Light">

<!-- Row 0 = input area, Row 1 = the matrix -->
<Grid RowDefinitions="Auto,*" Margin="10">

    <!-- Input area -->
    <StackPanel Grid.Row="0" Orientation="Horizontal" Spacing="8" Margin="0,0,0,10">
        <TextBlock Text="Task:" VerticalAlignment="Center"/>
        <TextBox x:Name="TaskInput" Width="350"/>
        <CheckBox x:Name="ImportantBox" Content="Important"/>
        <CheckBox x:Name="UrgentBox" Content="Urgent"/>
        <Button Content="Add" IsDefault="True" Click="AddButton_Click"/>
    </StackPanel>

    <!-- The 2x2 matrix -->
    <Grid Grid.Row="1" RowDefinitions="*,*" ColumnDefinitions="*,*">

        <Border Grid.Row="0" Grid.Column="0" Background="LightGreen"
            Margin="4" Padding="8" CornerRadius="4">
            <DockPanel>
                <TextBlock DockPanel.Dock="Top" Text="DO (Important + Urgent)"
                    FontWeight="Bold" Margin="0,0,0,6"/>
                <ListBox x:Name="DoList" DoubleTapped="List_DoubleTapped"/>
            </DockPanel>
        </Border>

        <Border Grid.Row="0" Grid.Column="1" Background="LightBlue"
            Margin="4" Padding="8" CornerRadius="4">
            <DockPanel>
                <TextBlock DockPanel.Dock="Top" Text="PLAN (Important + Not urgent)"
                    FontWeight="Bold" Margin="0,0,0,6"/>
                <ListBox x:Name="PlanList" DoubleTapped="List_DoubleTapped"/>
            </DockPanel>
        </Border>

        <Border Grid.Row="1" Grid.Column="0" Background="Khaki"
            Margin="4" Padding="8" CornerRadius="4">
            <DockPanel>
                <TextBlock DockPanel.Dock="Top" Text="DELEGATE (Not important + Urgent)"
                    FontWeight="Bold" Margin="0,0,0,6"/>
                <ListBox x:Name="DelegateList" DoubleTapped="List_DoubleTapped"/>
            </DockPanel>
        </Border>

        <Border Grid.Row="1" Grid.Column="1" Background="LightPink"
            Margin="4" Padding="8" CornerRadius="4">
            <DockPanel>
                <TextBlock DockPanel.Dock="Top" Text="DELETE (Not important + Not urgent)"
                    FontWeight="Bold" Margin="0,0,0,6"/>
                <ListBox x:Name="DeleteList" DoubleTapped="List_DoubleTapped"/>
            </DockPanel>
        </Border>

    </Grid>
</Grid>
</Window>

```

Here is how the layout works:

- **Grid** divides space into rows and columns. ***** means "take the remaining space," so ***,*** makes two equal halves, which gives you the 2x2 matrix.
- **x:Name** gives a control a name you can use from C#.
- **Click** and **DoubleTapped** connect events to methods you'll write in the next step.
- **IsDefault="True"** makes pressing Enter click the Add button.

- **RequestedThemeVariant="Light"** forces a light theme, so the text stays readable on the colored backgrounds even if your desktop uses dark mode.

Step 4: Write the logic in MainWindow.axaml.cs

Replace the entire contents of **MainWindow.axaml.cs** with this:

```
using System.Collections.ObjectModel;
using Avalonia.Controls;
using Avalonia.Input;
using Avalonia.Interactivity;

namespace EisenhowerMatrix;

public partial class MainWindow : Window
{
    // One collection of tasks per quadrant.
    // ObservableCollection automatically updates the ListBox when items change.
    private readonly ObservableCollection<string> doTasks = new();
    private readonly ObservableCollection<string> planTasks = new();
    private readonly ObservableCollection<string> delegateTasks = new();
    private readonly ObservableCollection<string> deleteTasks = new();

    public MainWindow()
    {
        InitializeComponent();

        // Connect each ListBox to its collection
        DoList.ItemsSource = doTasks;
        PlanList.ItemsSource = planTasks;
        DelegateList.ItemsSource = delegateTasks;
        DeleteList.ItemsSource = deleteTasks;
    }

    private void AddButton_Click(object? sender, RoutedEventArgs e)
    {
        string text = TaskInput.Text?.Trim() ?? "";

        if (text.Length == 0)
            return; // ignore empty tasks

        bool important = ImportantBox.IsChecked == true;
        bool urgent = UrgentBox.IsChecked == true;

        GetQuadrant(important, urgent).Add(text);

        // Reset the inputs for the next task
        TaskInput.Text = "";
        ImportantBox.IsChecked = false;
        UrgentBox.IsChecked = false;
        TaskInput.Focus();
    }

    // The Eisenhower rules
    private ObservableCollection<string> GetQuadrant(bool important, bool urgent)
    {
        return (important, urgent) switch
        {
            (true, true) => doTasks, // DO
            (true, false) => planTasks, // PLAN
            (false, true) => delegateTasks, // DELEGATE
            (false, false) => deleteTasks // DELETE
        };
    }

    // Double-click a task to remove it
}
```

```
private void List_DoubleTapped(object? sender, TappedEventArgs e)
{
    if (sender is ListBox list &&
        list.SelectedItem is string task &&
        list.ItemsSource is ObservableCollection<string> tasks)
    {
        tasks.Remove(task);
    }
}
}
```

Avalonia uses the `x:Name` values from the XAML to create fields automatically, which is why you can write `TaskInput` and `DoList` directly in C#.

Step 5: Run it

```
dotnet run
```

Type a task, tick the boxes that apply, and press Enter or click Add. The task appears in the correct quadrant, and double-clicking a task removes it.

The template's `Program.cs` and `App.axaml` already open `MainWindow`, so you don't need to change them.

Ideas for next steps

- **Confirm before removing.** Avalonia has no built-in `MessageBox`, but the `MessageBox.Avalonia` NuGet package provides one.
- **Save and load tasks.** Write the four collections to a JSON file with `System.Text.Json` when the window closes (the `Closing` event), and read them back in the constructor.
- **Move tasks between quadrants.** Add a `ContextMenu` to each `ListBox` in the XAML.
- **Distribute the app.** Build a standalone Linux executable with `dotnet publish -c Release -r linux-x64 --self-contained`.